

Documento de Arquitectura de Software (SAD)

Sistema de Gestión de Pedidos Multicanal & Control de Inventario

Proyecto: Leofit Solutions

1. Identificación y Control del Documento

Campo	Detalle
Institución:	Universidad Tecnológica del Perú (UTP)
Facultad:	Facultad de Ingeniería de Sistemas e Informática
Curso:	Integrador II: Software
Proyecto:	Sistema Web Progresivo (PWA) de Gestión de Pedidos & Control de Inventario en Tiempo Real
Empresa Beneficiaria:	Leofit (PYME textil deportiva - confección y comercialización)
Dueño de la Empresa:	Víctor Leandro Cárdenas Fernández
Versión del Documento:	1.0.0
Fecha:	Septiembre 2026

Equipo de Desarrollo (Grupo 01):

- 1. Cárdenas Fernández Víctor Leandro — Back-End / Base de Datos
- 2. Dávila Morales Jim Alessandro — Calidad / DevOps / Despliegue
- 3. Roman Delgado Harley Anthony — UX / Front-End
- 4. Loayza Rodriguez Lady Luz — UX / Front-End / Coordinación
- 5. Rojas Sanchez Daniel Enrique — Gestión / Análisis Funcional

2. Introducción y Objetivos Arquitectónicos

2.1. Propósito

El presente documento describe la arquitectura técnica integral, los patrones de diseño, las vistas estructurales (modelo 4+1) y las decisiones de ingeniería (ADR) adoptadas para la construcción del sistema de información de **Leofit**.

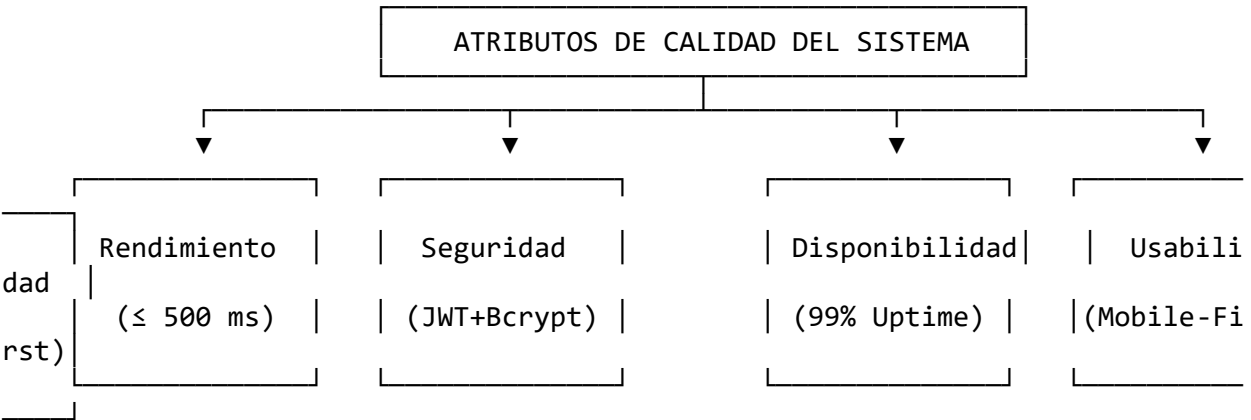
2.2. Contexto del Negocio y Problemática

Leofit gestionaba la recepción de pedidos a través de múltiples canales desarticulados (WhatsApp personal, llamadas telefónicas, mensajes directos de Instagram) registrando la información en cuadernos físicos o notas sueltas. Esta operativa generaba: * Sobreventa de prendas sin stock real disponible en el taller/almacén. * Retrasos y confusiones en el empaque y despacho de pedidos. * Cero trazabilidad del estado del pedido (Recibido → Preparación → En Camino → Entregado). * Falta de métricas consolidadas sobre el flujo de caja e ingresos diarios.

2.3. Objetivos Arquitectónicos

- 1. **Desacoplamiento Estricto:** Separación total entre la capa de interfaz de usuario (Frontend PWA) y la lógica de negocio/persistencia (API REST + Base de Datos).
- 2. **Alta Eficiencia & Baja Latencia:** Respuestas de consulta en menos de 500 ms y compilación ultrarrápida del cliente.
- 3. **Portabilidad y Soporte Mobile-First:** Experiencia ergonómica optimizada para smartphones con capacidad de instalación como Progressive Web App (PWA).
- 4. **Atomicidad e Integridad de Inventario:** Garantía ACID estricta en la reserva y descuento de prendas por talla y color.
- 5. **Seguridad y Trazabilidad:** Control de acceso basado en roles (RBAC) y auditoría de cambios de estado.

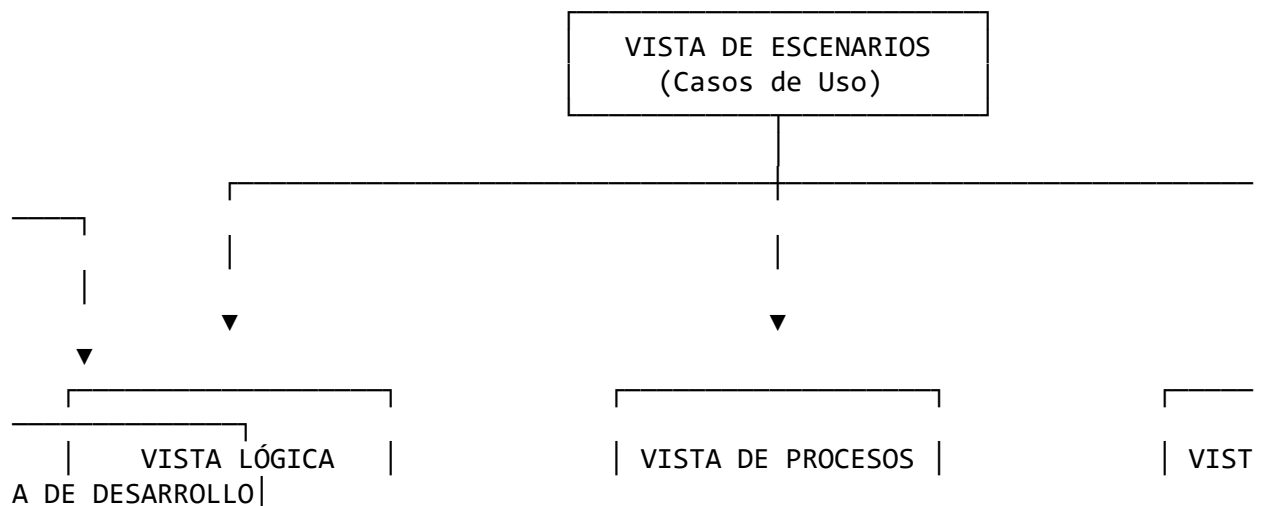
3. Atributos de Calidad (Requerimientos No Funcionales)

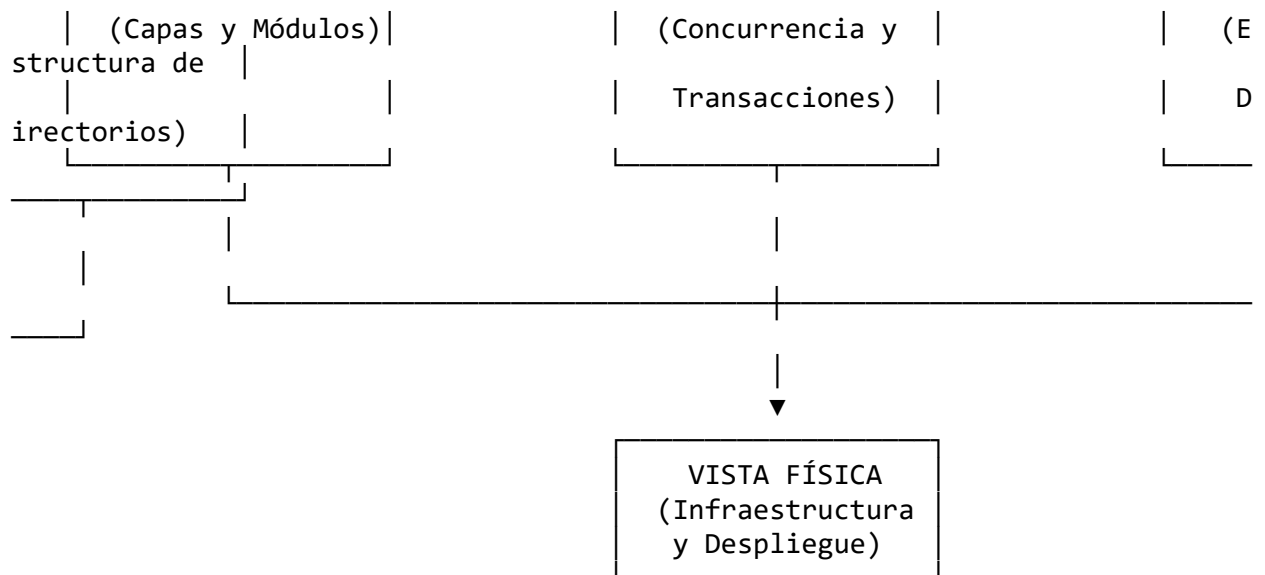


Atributo	Requerimiento / Métrica	Estrategia de Implementación
Rendimiento	Tiempo de carga inicial < 1.5s; tiempo de respuesta API ≤ 500 ms.	Compilación con Vite, bundle modular, minificación con Rollup, índices en base de datos.

Atributo	Requerimiento / Métrica	Estrategia de Implementación
Seguridad	Autenticación stateless y cifrado robusto de credenciales.	Tokens JWT firmados, contraseñas hasheadas con bcrypt (10 rounds), validación de inputs con schemas, CORS estricto.
Disponibilidad	99% de disponibilidad en horario comercial (08:00 - 22:00).	Despliegue en infraestructuras PaaS/Cloud de alta disponibilidad (GitHub Pages / Vercel + Render/Railway).
Escalabilidad	Capacidad de soportar picos de venta de temporada (campanas deportivas).	Arquitectura stateless en API REST que permite replicación horizontal de contenedores.
Mantenibilidad	Código limpio, tipado estático y pruebas automatizadas.	TypeScript estricto en frontend y backend, arquitectura en capas, pruebas con Vitest y CI/CD en GitHub Actions.
Usabilidad (UX)	Curva de aprendizaje < 15 minutos para operadores nuevos.	Enfoque mobile-first, diseño limpio con TailwindCSS, contraste accesible (WCAG AA) y retroalimentación visual inmediata.

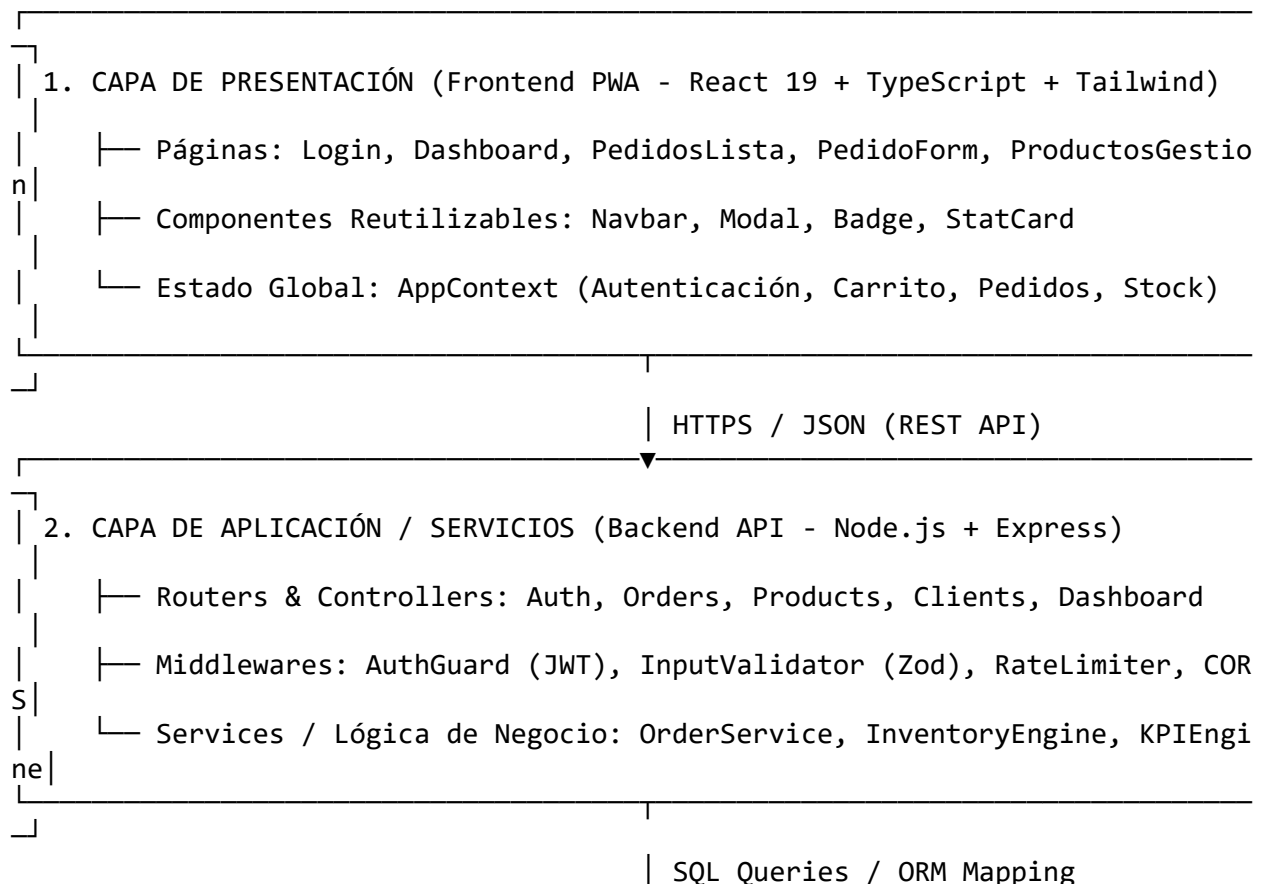
4. Vistas Arquitectónicas (Modelo 4+1 de Kruchten)

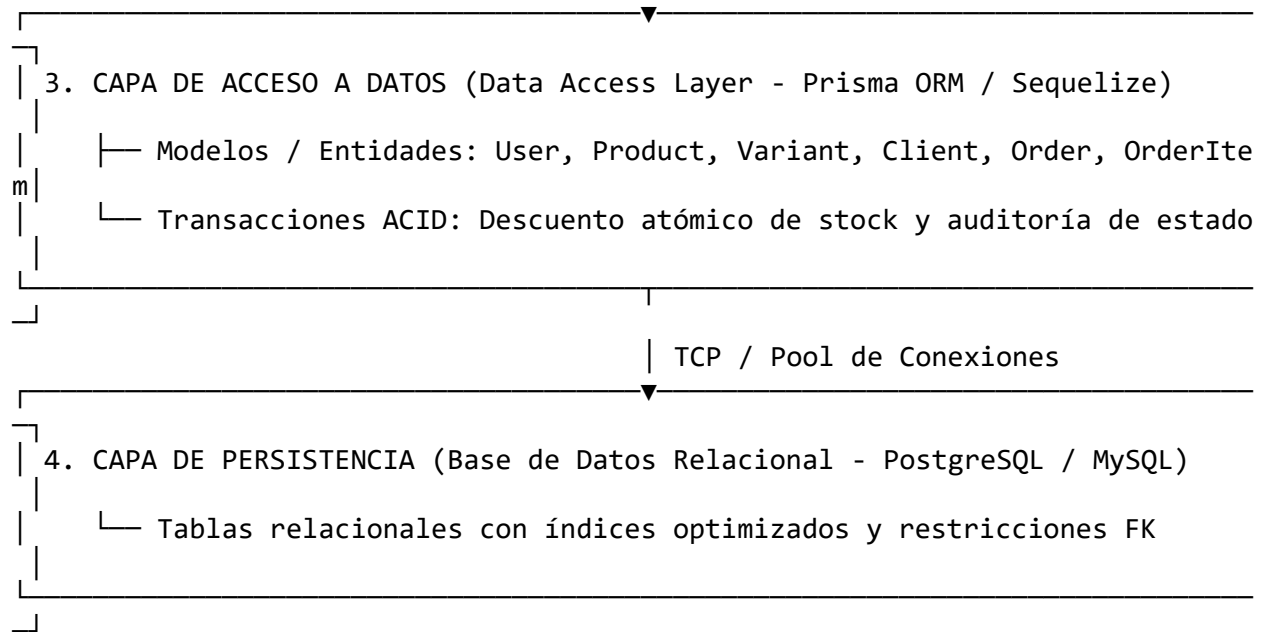




4.1. Vista Lógica: Arquitectura Multicapa

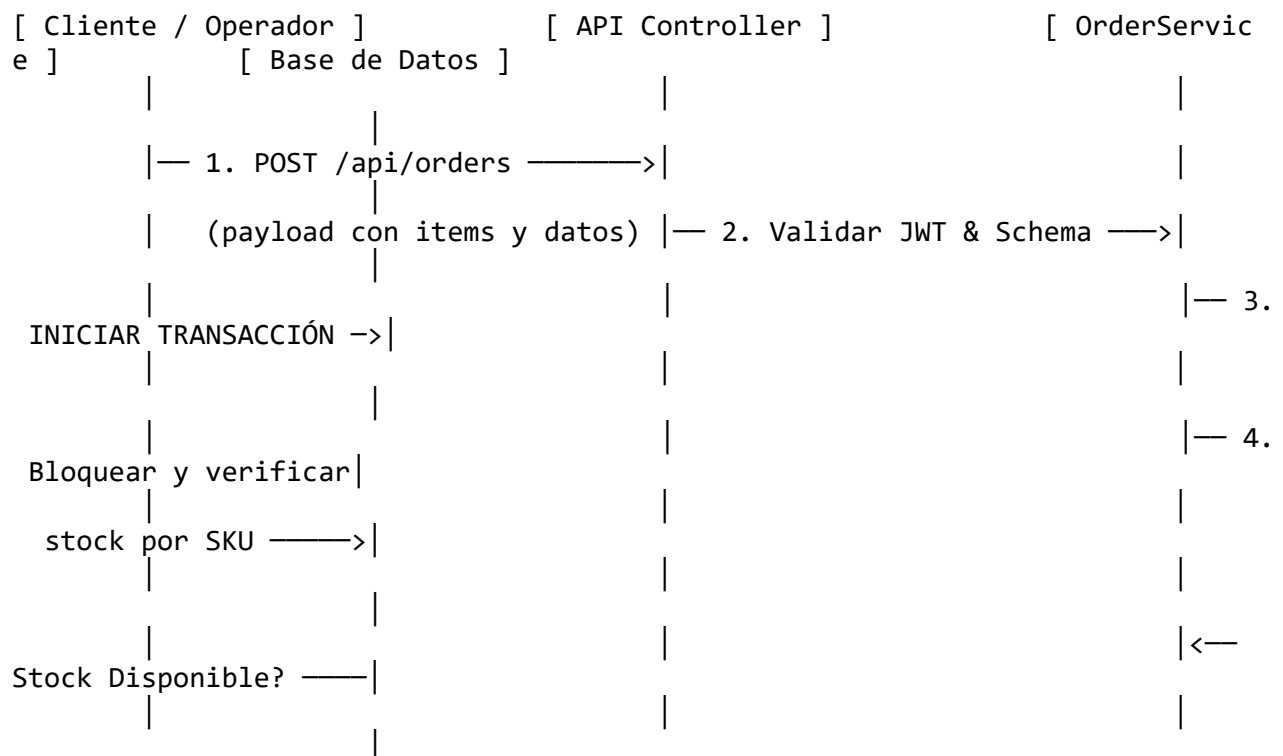
El sistema implementa una arquitectura desacoplada en cuatro niveles principales:

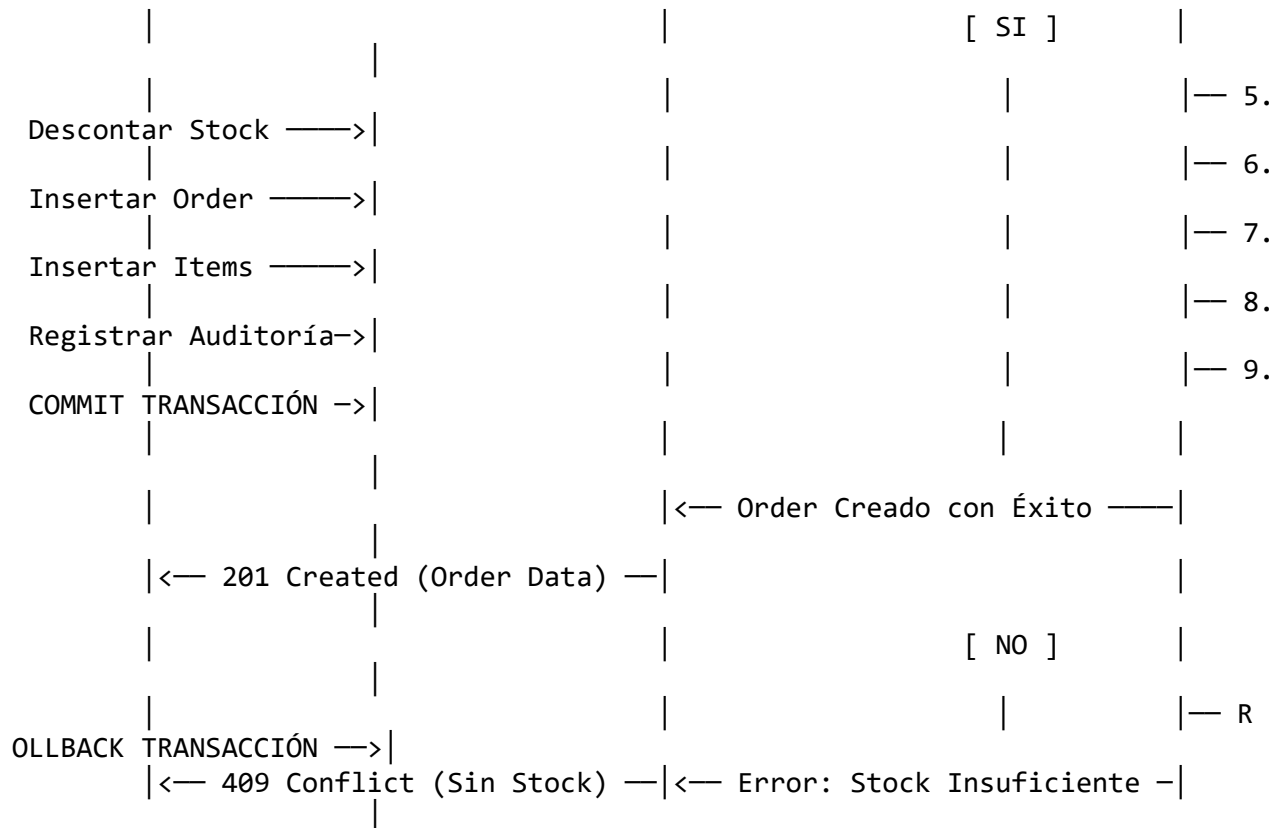




4.2. Vista de Procesos: Transaccionalidad Concurrente de Pedidos

Uno de los requerimientos más críticos es asegurar que no se vendan prendas que ya no tienen existencias físicas en el almacén. El siguiente flujo detalla el procesamiento concurrente y atómico:





4.3. Vista de Desarrollo: Estructura Modular del Proyecto

La estructura del código fuente está organizada bajo una arquitectura modular y mantenible:

leofit-pedidos-sistema/	
├─ package.json	# Configuración raíz de scripts y workspaces de ejecución
├─ index.html	# Punto de entrada SPA compilado para GitHub Pages
├─ assets/	# Bundle optimizado de JavaScript (257 KB) y CSS (32 KB)
├─ docs/	# Documentación formal (SAD, ERS, Glosario, Ficha, Actas)
├─ diagrams/	# Diagramas de arquitectura, BPMN, riesgos y matriz RF/RNF
├─ database/	# Modelo Entidad-Relación, esquemas y scripts SQL
├─ backend/	# Arquitectura y diseño de la API REST Node.js/Express
├─ frontend/	# Código fuente de la aplicación cliente (React 19 + Vite)
└─ public/	# Manifest PWA, iconos y manejador 404 SPA



MANAGED DATABASE
(PostgreSQL / MySQL)
Multi-AZ + Backups

4.5. Vista de Escenarios (+1 Casos de Uso Críticos)

Caso de Uso	Realización Arquitectónica	Componentes Involucrados
CU-01: Autenticación con Roles	Verificación de credenciales con hash Bcrypt; generación de token JWT con expiración; renderizado condicional por rol (Dueño vs Operador).	AuthService, Login.tsx, AppContext, AuthGuardMiddleware.
CU-02: Registro Ágil de Pedido	Cálculo reactivo de totales en cliente; petición POST atómica; validación de inventario en backend; decremento de stock; emisión de ticket formateado para WhatsApp.	PedidoForm.tsx, OrderController, InventoryEngine, database.orders.
CU-03: Transición de Estados	Máquina de estados finitos (Recibido → En Preparación → En Camino → Entregado); inserción de log en tabla de auditoría; actualización en tiempo real en la lista.	PedidosLista.tsx, OrderService, order_status_history.
CU-04: Alerta de Stock Crítico	Evaluación de umbral (stock < 5); inyección de badges visuales pulsantes; bloqueo preventivo de pedidos que excedan el saldo disponible.	ProductosGestion.tsx, Dashboard.tsx, product_variants.

5. Registro de Decisiones de Arquitectura (ADR)

ADR-01: Adopción de Arquitectura Desacoplada (PWA + API REST)

- **Estado:** Aceptado.

- **Contexto:** Se requería una solución que permita atender pedidos desde el celular en el taller y desde computadoras en el punto de venta.
- **Decisión:** Implementar una Progressive Web App (PWA) construida con React 19 + TypeScript, consumiendo una API REST desacoplada.
- **Consecuencias Positivas:** Independencia de despliegue, experiencia nativa en móviles sin coste de tiendas de aplicaciones, reutilización de la API para futuras integraciones.

ADR-02: Selección de Base de Datos Relacional (SQL)

- **Estado:** Aceptado.
- **Contexto:** Las transacciones de venta e inventario requieren estricta consistencia para no permitir sobreventas.
- **Decisión:** Utilizar un motor relacional (PostgreSQL / MySQL) con soporte de claves foráneas e índices B-Tree.
- **Consecuencias Positivas:** Cumplimiento total de propiedades ACID; integridad referencial garantizada; facilidad para reportes contables.

ADR-03: Autenticación Stateless con JSON Web Tokens (JWT)

- **Estado:** Aceptado.
- **Contexto:** Se necesita un esquema de autenticación seguro, ligero y que no dependa de sesiones en memoria en el servidor.
- **Decisión:** Emisión de tokens JWT firmados con algoritmo HMAC-SHA256, con payload que incluye userId y role.
- **Consecuencias Positivas:** Escalabilidad horizontal sin replicación de sesiones; consumo seguro desde navegadores y dispositivos móviles.

ADR-04: Enfoque de Pruebas Automatizadas con Vitest

- **Estado:** Aceptado.
- **Contexto:** Validación rápida de las reglas de cálculo (subtotales, descuentos, cálculo de riesgo de pedidos) en el flujo de CI.
- **Decisión:** Utilizar Vitest integrado de forma nativa con Vite.
- **Consecuencias Positivas:** Ejecución de pruebas unitarias en menos de 200 ms sin sobrecarga de configuración de Babel/Jest.

6. Estrategia de Seguridad y Gobernanza

1. Gestión de Variables de Entorno:

- Cero secretos en código fuente. Parámetros sensibles (JWT_SECRET, DB_URL, credenciales) inyectados mediante archivos .env excluidos del control de versiones.

2. Control de Acceso Basado en Roles (RBAC):

- **Rol Administrador (Dueño):** Acceso total a creación de pedidos, edición de precios, control de inventario y visualización de montos de facturación.
- **Rol Operador:** Acceso restringido a cambio de estados operativos y visualización de datos de entrega, con ocultamiento de métricas financieras.

3. Pipeline de Integración y Despliegue Continuo (CI/CD):

- Validación automática de linting, tipado estático, suite de pruebas unitarias y escaneo de vulnerabilidades (`security-scan.yml`) ante cada Pull Request hacia la rama `main`.